

Final project NLP

Goryunov Kirill, Mochalov Ilya
Higher School of Economics, Faculty of Computer Science
(Dated: Mar 26, 2026)

The source code of the project can be viewed on github: <https://github.com/randnull/nlp>

I. INTRODUCTION

This work aims to research an BIO and BIOES labeling schemes for transformer-based named entity recognition.

Albert is going to United States of America.

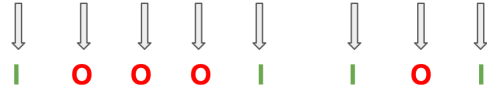


FIG. 2: IO tagging on example

II. DATASET

The *conll2003 dataset* [1] was selected as main dataset for this work. This dataset contains more than 14,000 samples with ner tags.

The main weakness of the IO scheme is its inability to distinguish between two adjacent entities of the same type. For example, in the phrase "*John Smith Jane Doe met*" would be four I-Person tokens but we have two different persons on fact.

III. NER

A. IO

First possible tagging algorithm is IO. This is one of the simplest and most basic algorithms, which only involves dividing into basic parts. More details can be found in the image 1.

It consists of two tags

- I-(Inside): A token that belongs to an entity (for example, I-PER for any token of a person name);
- O (Outside): A token that does not refer to any entity.

B. BIO

The task of BIO classification is to determine the meaning of a word in a sentence like in IO but in extend form.

In addition to IO tags there is another one - B tag (means Begin), we use it to understand if entity started. All next words in entity will be mentioned by I (Inside) tag.

- B-(Begin): The beginning of an entity (for example, B-PER for the beginning of a name);
- I-(Inside): The internal part of an entity (for example, I-PER for the continuation of a name);
- O (Outside): A token that does not refer to any entity.

Here is an example of sentence tags 3.

Words	IO Label	BIO Label	BIOES Label
Jane	I-PER	B-PER	B-PER
Villanueva	I-PER	I-PER	E-PER
of	O	O	O
United	I-ORG	B-ORG	B-ORG
Airlines	I-ORG	I-ORG	I-ORG
Holding	I-ORG	I-ORG	E-ORG
discussed	O	O	O
the	O	O	O
Chicago	I-LOC	B-LOC	S-LOC
route	O	O	O
.	O	O	O

FIG. 1: NER Tagging

In the picture you can see an example 2.

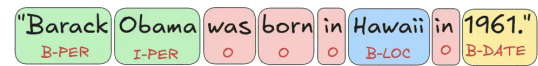


FIG. 3: BIO tagging on example

C. BIOES

BIOES uses a wide variety of different labels. In addition to BIO, it uses S (Single) to identify a single entity.

It also has a different meaning for the tokens B (Begin) - the beginning of a multi-word entity and E (End) - the end of a multi-word entity. All tokens within this entity are marked as I (Inside).

- B-(Begin): The beginning of a multi-word entity (for example, B-PER for the first token of a name);
- I-(Inside): The internal part of a multi-word entity (for example, I-PER for a middle token of a name);
- O (Outside): A token that does not refer to any entity;
- E-(End): The last token of a multi-word entity (for example, E-PER for the last token of a name);
- S-(Single): A single-token entity (for example, S-LOC for a one-word location).

This way the model receives a stronger signal about entity boundaries: every span has both an explicit start and an explicit end and single-word entities are clearly separated from the first token of longer ones.

In the picture you can see an example 4.



FIG. 4: BIOES tagging on example

IV. TOKENIZATION

For the tokenization task we will also use `bert-base-cased`.

Tokenization causes problems due to the fact that $\text{len}(\text{tokens}) \geq \text{len}(\text{words})$, and we only have markup for words. The problem is illustrated in more detail in the image 5.

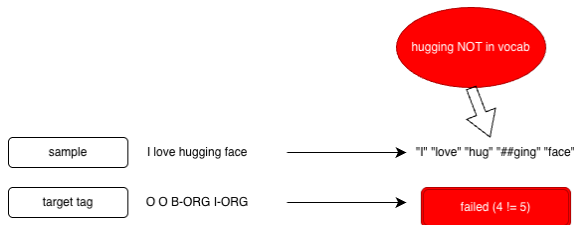


FIG. 5: BIO

To solve this problem, we can either ignore subsequent parts or count them as part of the word (continuation).

In our work we will take them into account, described in detail in the image

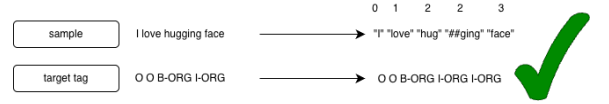


FIG. 6: BIO

V. TRANSFORMER

In order to use tagging, we will use the BERT encoder-decoder transformer model. To do this, we will use a ready-made BERT model from the transformer library. We will use `AutoModelForTokenClassification` due to our task is classificate every token, not all sentence (otherwise we need `AutoModelForSequenceClassification`)

Listing 1: Importing tokenizer and token classification model from Transformers

```
1 from transformers import AutoTokenizer,
   AutoModelForTokenClassification
```

We will use `bert-base-cased` as main transformer model. The key difference with uncased is that our model will be all case sensitive (upper and lower case letters). This is very useful for our task, because words can have different meanings in different registers.

VI. METRICS

To assess the quality of the model, we will use the `seval` library [2], designed for sequence tagging tasks such as named entity recognition. It allows you to calculate precision, recall, F1-score and accuracy metrics based on a comparison of true and predicted token labels. In this work, `seval` is used to analyze how correctly the model determines the boundaries of named entities and assigns them the appropriate classes.

VII. RESULTS

TABLE II: Evaluation results for the BIO sequence labeling task.

Dataset	Model	Tokenizer	F1	Acc.
CoNLL-2003	<code>bert-base-cased</code>	<code>bert-base-cased</code>	0.8976	0.9709

TABLE I: Per-token predictions. **Green** = correct, **red** = error.

Word	Gold	IO	BIO	BIOES
<i>“Nadim Ladki” (PER)</i>				
Nadim	B-PER	I-PER	B-PER	B-ORG
Ladki	I-PER	O	I-PER	E-ORG
<i>“AL-AIN , United Arab Emirates” (LOC)</i>				
AL-AIN	B-LOC	I-LOC	B-LOC	S-LOC
,	O	I-LOC	O	O
United	B-LOC	I-LOC	B-LOC	B-LOC
Arab	I-LOC	O	I-LOC	I-LOC
Emirates	I-LOC	O	I-LOC	E-LOC

VIII. CONCLUSION

In addition to empirical metrics, you can also look at the objective part of the problem. IO gives a simple classification - whether a word refers to any entity. For example, we want to get all the medical terms that were mentioned in the text - then we just look at I-TERM.

On the other hand, our task might be to extract the full names of drugs or organizations (multiwords). For these tasks you can use BIO or BIOES. BIOES is better at defining boundaries, but more complex (contains more tags)

[1] Huggingface, *conll2003*, <https://huggingface.co/datasets/tomaarsen/conll2003> (accessed: 25 April 2026).

[2] Sequeval library, *sequeval*, <https://pypi.org/project/sequeval/0.0.10> (accessed: 27 April 2026).